

End-to-End DevOps CI/CD Pipeline Project Documentation

Project Title

End-to-End DevOps CI/CD Pipeline for Customer Support Ticket System Using MERN Stack, Jenkins, Docker, Docker Hub, Kubernetes, SonarQube, Prometheus, Grafana, and AWS EC2

Project Overview

This project demonstrates a complete **DevOps CI/CD pipeline** implemented on an AWS EC2 instance using industry-standard DevOps tools and practices. The pipeline automates the process of source code integration, code quality analysis, containerization, image management, Kubernetes deployment, and application monitoring.

The project uses **GitHub** for source code management, **Jenkins** for Continuous Integration and Continuous Deployment (CI/CD), **SonarQube** for code quality analysis, **Docker** for containerization, **Docker Hub** for image storage, **Kubernetes (Minikube)** for container orchestration, and **Prometheus** and **Grafana** for monitoring and visualization.

Whenever code is pushed to GitHub:

1. GitHub triggers the Jenkins pipeline automatically.
2. Jenkins checks out the latest source code.
3. SonarQube performs code quality analysis.
4. Docker images are built for the application.
5. Docker images are pushed to Docker Hub.
6. Kubernetes pulls the latest images and deploys the updated application.
7. Prometheus collects application and cluster metrics.
8. Grafana displays real-time monitoring dashboards.

This project demonstrates:

- CI/CD pipeline automation
- Source code management using GitHub
- Code quality analysis with SonarQube
- Docker containerization
- Docker Hub image management
- Kubernetes deployment and orchestration
- Automated application deployment
- Infrastructure and application monitoring using Prometheus
- Real-time visualization with Grafana
- AWS EC2 deployment

Technologies Used

Technology	Purpose
Git	Version control and source code management
GitHub	Remote repository hosting and source code management
Jenkins	Continuous Integration and Continuous Deployment (CI/CD) automation
Node.js	Runtime environment for the application
React.js	Frontend user interface
Docker	Containerization of the application
Docker Hub	Container image registry
Kubernetes (Minikube)	Container orchestration and application deployment
kubectl	Kubernetes command-line management tool
Helm	Package manager for Kubernetes
SonarQube	Code quality and static code analysis
Prometheus	Monitoring and metrics collection
Grafana	Metrics visualization and dashboard monitoring
AWS EC2	Cloud virtual machine hosting the complete environment
Linux (Amazon Linux 2023)	Operating system used for deployment

Scope of the Project

The scope of this project is to implement a complete **DevOps CI/CD pipeline** that automates the software delivery process from source code management to deployment and monitoring. The project demonstrates the integration of modern DevOps tools to improve development efficiency, deployment reliability, and system monitoring.

The project includes:

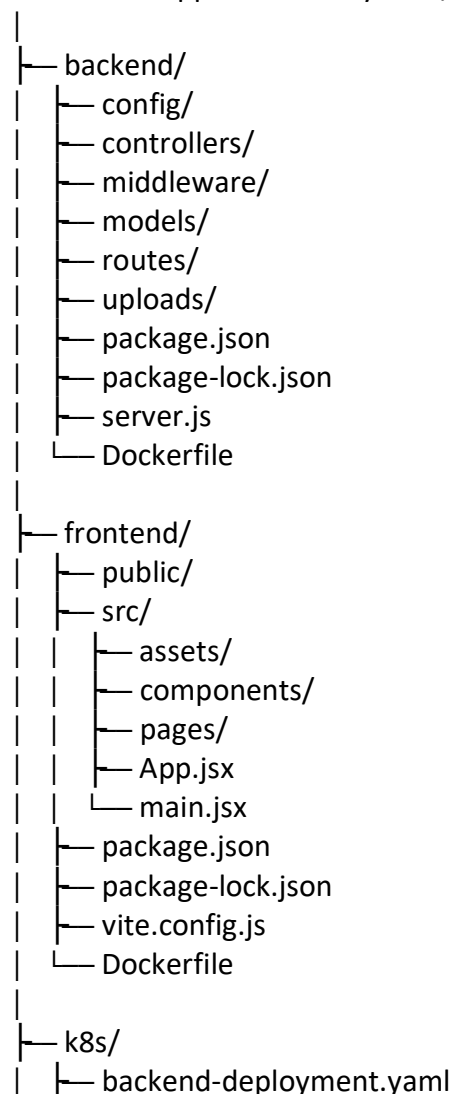
- Source code management using Git and GitHub.
- Automated Continuous Integration and Continuous Deployment (CI/CD) using Jenkins.
- Code quality analysis using SonarQube.
- Containerization of the application using Docker.
- Storage and management of Docker images using Docker Hub.
- Deployment and orchestration using Kubernetes (Minikube).
- Real-time monitoring using Prometheus.
- Visualization of system and application metrics using Grafana.
- Deployment and management of the complete environment on an AWS EC2 instance.
- Automated application updates whenever new code is pushed to the GitHub repository.

This Project Covers

- Git and GitHub for Source Code Management
- Jenkins for Continuous Integration and Continuous Deployment (CI/CD)
- Automated Pipeline Triggering from GitHub
- Dependency Installation and Build Automation
- Code Quality Analysis using SonarQube
- Docker Containerization
- Docker Image Creation and Management
- Docker Hub Integration
- Kubernetes (Minikube) Deployment
- Kubernetes Deployments and Services
- Automated Application Deployment
- Application Scaling using Kubernetes
- Prometheus for Monitoring and Metrics Collection
- Grafana for Dashboard Visualization
- Real-time Infrastructure and Application Monitoring
- AWS EC2 Deployment Environment
- End-to-End DevOps CI/CD Workflow
- Automated Software Delivery Pipeline

Folder Structure:

Customer-Support-Ticket-System/



```

├── backend-service.yaml
├── frontend-deployment.yaml
├── frontend-service.yaml
├── Jenkinsfile
├── README.md
├── .gitignore
└── LICENSE

```

Implementation Steps

Step 1: Clone the Project from GitHub

The Customer Support Ticket System source code was cloned from the GitHub repository to the AWS EC2 instance. This repository contains the frontend, backend, Kubernetes manifests, and Jenkins pipeline configuration required for implementing the DevOps workflow.

```
git clone https://github.com/mudasiryousuf/Customer-Support-Ticket-System.gitcd Customer-Support-Ticket-Systemls
```

```

[ec2-user@ip-172-31-14-243 ~]$ ls
Customer-Support-Ticket-System  minikube-linux-amd64
[ec2-user@ip-172-31-14-243 ~]$

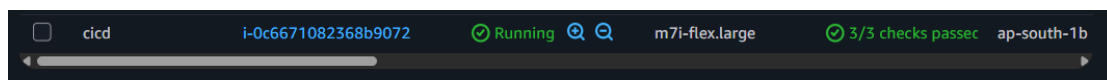
```

Step 2: Launch AWS EC2 Instance

An **Amazon EC2** instance was created to host the complete DevOps environment including Jenkins, Docker, Kubernetes (Minikube), SonarQube, Prometheus, and Grafana.

EC2 Configuration

Property	Value
Instance Type	t3.large
Operating System	Amazon Linux 2023
Storage	30 GB
Architecture	x86_64



Step 3: Configure Security Group

The EC2 security group was configured to allow access to the required services.

Port	Protocol	Purpose
22	TCP	SSH Access
8080	TCP	Jenkins
9000	TCP	SonarQube
3000	TCP	Grafana
9090	TCP	Prometheus
80	TCP	HTTP

Security group rule ID	Type	Info	Protocol	Info	Port range	Source	Info	Description - optional	Info
sgr-0f296a77a67c43220	HTTP		TCP		80	Cu...			Delete
sgr-05d480efd25849f27	Custom TCP		TCP		3000	Cu...		0.0.0.0/0	Delete
sgr-0d4d53a3d531e6e0c	SSH		TCP		22	Cu...		0.0.0.0/0	Delete
sgr-01407c4c06726ded3	Custom TCP		TCP		8080	Cu...		0.0.0.0/0	Delete

Step 4: Connect to the EC2 Instance

The EC2 instance was accessed securely using SSH.

Step 5: Update the System

The operating system packages were updated to ensure the latest security patches and software versions.

```
sudo dnf update -y
```

Step 6: Install Git

Git was installed to clone the project repository and manage source code.

```
sudo dnf install git -ygit --version
```

Step 7: Install Node.js and npm

Node.js and npm were installed to build the frontend and backend applications.

```
sudo dnf install nodejs -y
node -vnode -v
```

Step 8: Install Java

Java was installed because Jenkins and SonarQube require the Java Runtime Environment (JDK) to run.

```
sudo dnf install java-17-amazon-corretto -y
```

```
[ec2-user@ip-172-31-14-243 ~]$ java -version
openjdk version "21.0.11" 2026-04-21 LTS
OpenJDK Runtime Environment Corretto-21.0.11.10.1 (build 21.0.11+10-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.11.10.1 (build 21.0.11+10-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-14-243 ~]$
```

Step 9: Install Jenkins

Jenkins was installed to automate the Continuous Integration and Continuous Deployment (CI/CD) pipeline.

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
sudo dnf install jenkins -y
```

Step 10: Start and Enable Jenkins

After installation, the Jenkins service was started and configured to start automatically after every system reboot.

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

```
• jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-07-08 18:18:54 UTC; 20h ago
   Main PID: 141153 (java)
```

Step 11: Configure Jenkins Security Group

To access the Jenkins web interface, port **8080** was added to the EC2 Security Group as an inbound rule.

Port	Protocol	Source	Purpose
8080	TCP	Anywhere (0.0.0.0/0)	Jenkins Dashboard

Jenkins was accessed using:

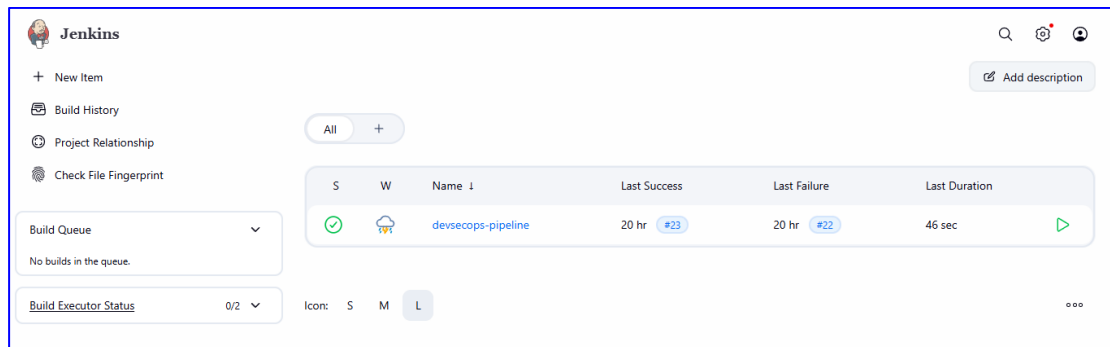
`http://<EC2-IP>:8080`

Step 12: Unlock Jenkins

The initial administrator password was retrieved from the EC2 instance to unlock Jenkins.

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

The password was copied and pasted into the Jenkins setup page.



Step 13: Install Suggested Plugins

After unlocking Jenkins:

- Click **Install Suggested Plugins**.

Step 14: Install Required Jenkins Plugins

The following plugins were installed from **Manage Jenkins → Plugins**:

- Docker
- NodeJS
- SonarQube Scanner
- Sonar Quality Gates
- Kubernetes CLI

Step 15: Configure Jenkins Tools

Navigate to:

Manage Jenkins → Tools

Configure the following tools:

- Git
- NodeJS
- SonarScanner

These tools are used by the Jenkins pipeline during the build and deployment process.

Step 16: Install Docker

Docker was installed to containerize the frontend and backend applications.

```
sudo dnf install docker -y
```

```
[ec2-user@ip-172-31-14-243 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-172-31-14-243 ~]$
```

Step 17: Start and Enable Docker

The Docker service was started and configured to start automatically during system boot.

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
sudo systemctl status docker
```

```
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-07-08 09:37:03 UTC; 1 day 5h ago
     TriggeredBy: ● docker.socket
```

Step 18: Configure Docker Permissions

The Jenkins and EC2 user were added to the Docker group to execute Docker commands without requiring root privileges.

```
sudo usermod -aG docker ec2-user
```

```
sudo usermod -aG docker jenkins
```

```
sudo systemctl restart docker
```

```
sudo systemctl restart jenkins
```

Verify Docker access:

```
docker ps
```

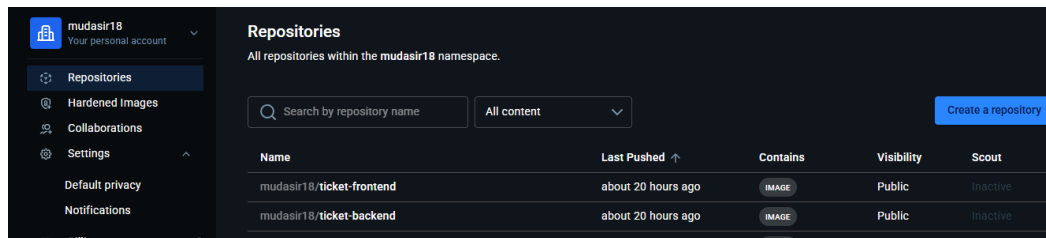
Step 19: Install Docker Buildx

Docker Buildx was installed to support modern Docker image builds.

```
docker buildx version
```

Step 20: Create Docker Hub Repository

A Docker Hub account was used to create two repositories for storing application images.



Docker Hub Username:

mudasir18

Step 21: Login to Docker Hub

Docker was authenticated with Docker Hub to push container images.

docker login

Enter:

- Docker Hub Username
- Docker Hub Password / Access Token

Step 22: Build Docker Images

Docker images were built separately for the backend and frontend applications.

Backend Image

docker build -t mudasir18/ticket-backend:v1 backend

Frontend Image

docker build -t mudasir18/ticket-frontend:v1 frontend

Verify images:

```
[ec2-user@ip-172-31-14-243 ~]$ docker images
REPOSITORY              TAG          IMAGE ID          CREATED          SIZE
mudasir18/ticket-frontend v1           158fe35e0d0b     30 hours ago    319MB
null/ticket-frontend     v1           158fe35e0d0b     30 hours ago    319MB
null/ticket-backend      v1           f30dcb491e4d     30 hours ago    172MB
```

Step 23: Push Docker Images to Docker Hub

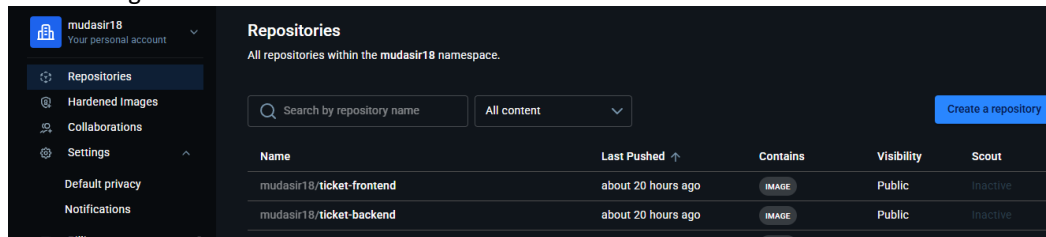
The Docker images were uploaded to Docker Hub.

docker push mudasir18/ticket-backend:v1

docker push mudasir18/ticket-frontend:v1

Verify:

docker images



Step 24: Install kubectl

The Kubernetes command-line tool (**kubectl**) was installed to manage the Kubernetes cluster.

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
```

```
kubectl version --client
```

Step 25: Install Minikube

Minikube was installed to create a local Kubernetes cluster on the AWS EC2 instance.

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

```
minikube version
```

Step 26: Start Minikube

The Kubernetes cluster was started using the Docker driver.

```
minikube start --driver=docker
```

Verify cluster status:

```
minikube status
```

```
kubectl get nodes
```

```
[ec2-user@ip-172-31-14-243 ~]$ kubectl get nodes
NAME           STATUS    ROLES          AGE    VERSION
minikube       Ready     control-plane   2d     v1.35.1
[ec2-user@ip-172-31-14-243 ~]$
```

This completes the Docker and Kubernetes installation phase. The next section will cover creating the Kubernetes manifests and deploying the application

Step 27: Create Kubernetes Manifest Files

A dedicated **k8s** directory was created to store the Kubernetes manifest files required for deploying the application.

```
mkdir k8s
```

The following manifest files were created:

- backend-deployment.yaml
- backend-service.yaml
- frontend-deployment.yaml
- frontend-service.yaml

```
[ec2-user@ip-172-31-14-243 Customer-Support-Ticket-System]$ ls
Jenkinsfile  backend  frontend  k8s
[ec2-user@ip-172-31-14-243 Customer-Support-Ticket-System]$ cd k8s/
[ec2-user@ip-172-31-14-243 k8s]$ ls
backend-deployment.yaml  backend-service.yaml  frontend-deployment.yaml  frontend-service.yaml
[ec2-user@ip-172-31-14-243 k8s]$
```

Step 28: Configure Backend Deployment

A Kubernetes Deployment was created for the backend application.

```
[ec2-user@ip-172-31-14-243 k8s]$ cat backend-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ticket-backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ticket-backend
  template:
    metadata:
      labels:
        app: ticket-backend
    spec:
      containers:
        - name: ticket-backend
          image: mudasir18/ticket-backend:v1
          ports:
            - containerPort: 5000
          env:
            - name: DB_HOST
              value: "mariadb"
            - name: DB_USER
              value: "ticketuser"
            - name: DB_PASSWORD
              value: "ticket123"
            - name: DB_NAME
              value: "ticket_system"
[ec2-user@ip-172-31-14-243 k8s]$
```

Step 29: Configure Backend Service

A Kubernetes **NodePort Service** was created to expose the backend application.

```
[ec2-user@ip-172-31-14-243 k8s]$ cat backend-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: ticket-backend
  ports:
    - port: 5000
      targetPort: 5000
  type: NodePort
[ec2-user@ip-172-31-14-243 k8s]$
```

Step 30: Configure Frontend Deployment

A Kubernetes Deployment was created for the frontend application.

```
[ec2-user@ip-172-31-14-243 k8s]$ cat frontend-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ticket-frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ticket-frontend
  template:
    metadata:
      labels:
        app: ticket-frontend
    spec:
      containers:
        - name: ticket-frontend
          image: mudasir18/ticket-frontend:v1
          ports:
            - containerPort: 5173
          env:
            - name: VITE_API_URL
              value: "http://backend-service:5000/api/tickets"
[ec2-user@ip-172-31-14-243 k8s]$
```

Step 31: Configure Frontend Service

A Kubernetes **NodePort Service** was created to expose the frontend application.

```
[ec2-user@ip-172-31-14-243 k8s]$ cat frontend-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: ticket-frontend
  ports:
    - port: 5173
      targetPort: 5173
  type: NodePort
[ec2-user@ip-172-31-14-243 k8s]$
```

Step 32: Deploy the Application to Kubernetes

The Kubernetes manifest files were applied to deploy the application.

```
kubectl apply -f k8s/
```

Step 33: Verify Kubernetes Resources

Verify that all resources were created successfully.

```
kubectl get deployments
kubectl get svc
kubectl get nodes
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running	4 (11m ago)	20h
monitoring-grafana-5bc65c4644-xkhp	3/3	Running	6 (11m ago)	20h
monitoring-kube-prometheus-operator-55c9dfccff-wm6tg	1/1	Running	4 (11m ago)	20h
monitoring-kube-state-metrics-6b8f7fb688-864bk	1/1	Running	3 (11m ago)	20h
monitoring-prometheus-node-exporter-582qs	1/1	Running	2 (11m ago)	20h
prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running	4 (11m ago)	20h
ticket-backend-5ffcbdbb49-6n6cr	1/1	Running	2 (11m ago)	20h
ticket-frontend-5bf98dffc6-xkmqj	1/1	Running	2 (11m ago)	20h

Expected Resources:

- Backend Deployment
- Frontend Deployment
- Backend Service
- Frontend Service
- Running Pods

Step 34: Access the Application

The application is deployed as Kubernetes Pods in a Minikube cluster running on an AWS EC2 instance. The frontend and backend are exposed through Kubernetes NodePort Services and can be accessed for testing using Minikube service URLs or Kubernetes port forwarding.

Retrieve the service URL using Minikube.

```
kubectl get svc
```

Alternatively,

```
minikube service frontend-service --url
```

To access it from your laptop browser

```
kubectl port-forward svc/frontend-service 5173:5173 --address 0.0.0.0
```

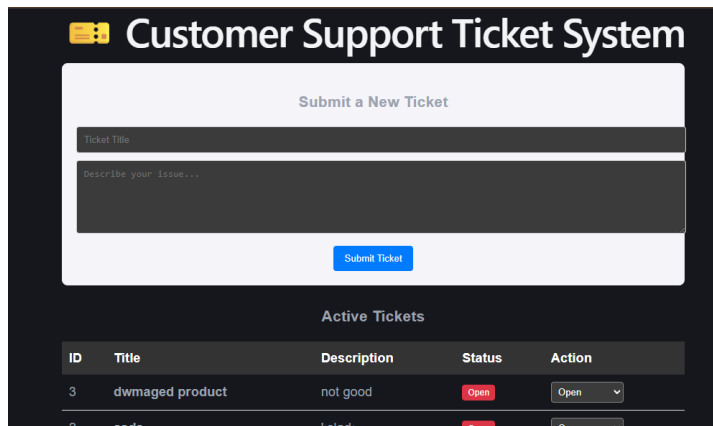
Then add an inbound rule in your EC2 Security Group:

- **Custom TCP**
- **Port:** 5173
- **Source:** 0.0.0.0/0 (or your IP for better security)

```
[ec2-user@ip-172-31-14-243 k8s]$ minikube service frontend-service --url
http://192.168.49.2:32263
[ec2-user@ip-172-31-14-243 k8s]$ kubectl port-forward svc/frontend-service 5173:5173 --address 0.0.0.0
Forwarding from 0.0.0.0:5173 -> 5173
```

Access the application using:

<http://ec2pubip:5173>



Step 35: Install SonarQube

SonarQube Community Edition was deployed as a Docker container for code quality analysis.

```
docker run -d \
  --name sonarqube \
  -p 9000:9000 \
  sonarqube:lts-community
```

Verify the container:

```
docker ps
[ec2-user@ip-172-31-14-243 ~]$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
ae3dd23ee78c   sonarqube:lts-community            "/opt/sonarqube/dock... 22 hours ago   Up 22 hours   0.0.0.0:9000->9000/tcp, :::9000->9000/tcp
sonarqube
abbb6c3a0832   gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr... 2 days ago     Up 30 hours   127.0.0.1:32770->5000/tcp, 127.0.0.1:32769->8443/tcp, 127.0.0.1:32768->32443/tcp
minikube
[ec2-user@ip-172-31-14-243 ~]$
```

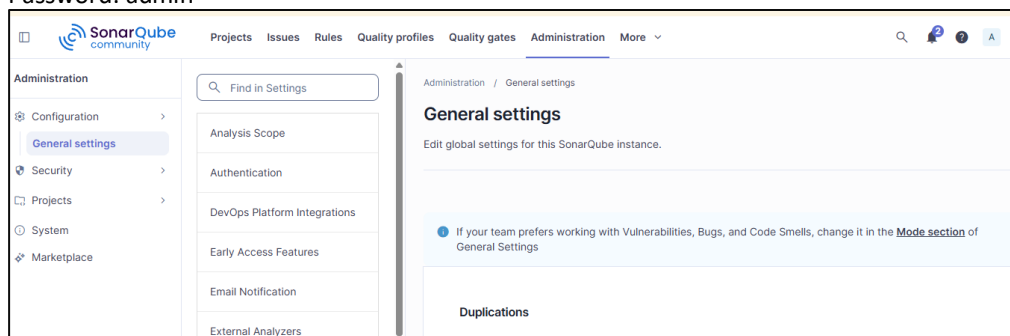
Open SonarQube:

<http://<EC2-PUBLIC-IP>:9000>

Default credentials:

Username: admin

Password: admin



Step 36: Configure SonarQube

After logging in:

- Change the default password.
- Generate a project token.
- Save the generated token for Jenkins integration.

Step 37: Configure SonarQube in Jenkins

Navigate to:

Manage Jenkins → System

Configure:

- SonarQube Server Name
- SonarQube Server URL
- Authentication Token

Then navigate to:

Manage Jenkins → Tools

Configure:

- SonarScanner Installation

This completed the integration between Jenkins and SonarQube for automated code quality analysis.

Step 38: Configure Jenkins Credentials

Docker Hub and SonarQube credentials were securely stored in Jenkins using the **Credentials Manager**.

Navigate to:

Manage Jenkins → Credentials

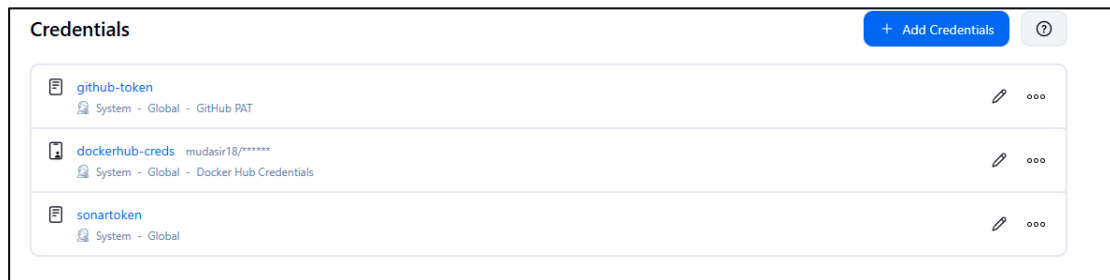
Add the following credentials:

Docker Hub Credentials

- Kind: **Username with Password**
- ID: dockerhub-creds
- Username: Docker Hub Username
- Password: Docker Hub Password/Access Token

SonarQube Token

- Kind: **Secret Text**
- ID: sonarqube-token
- Secret: SonarQube Project Token



Step 39: Configure NodeJS and SonarScanner in Jenkins

Navigate to:

Manage Jenkins → Tools

Configure the following tools:

- NodeJS Installation
- SonarScanner Installation

These tools are used by the Jenkins pipeline during application build and code quality analysis.

Step 40: Create Jenkins Pipeline Job

A Pipeline project was created in Jenkins.

Navigate to:

Dashboard → New Item

- Item Name: devsecops-pipeline
- Project Type: **Pipeline**

Pipeline Configuration:

- Definition: **Pipeline script from SCM**
- SCM: **Git**
- Repository URL:

<https://github.com/mudasiryousuf/Customer-Support-Ticket-System.git>

- Branch:

main

- Script Path:

Jenkinsfile

Save the configuration.

Step 41: Create the Jenkins Pipeline

A **Jenkinsfile** was created in the project root directory to automate the CI/CD workflow.

```
stage('Install Frontend Dependencies') {
  steps {
    dir('frontend') {
      sh 'npm install'
    }
  }
}

stage('SonarQube Scan') {
  steps {
    script {
      def scannerHome = tool 'SonarScanner'

      withSonarQubeEnv('SonarQube') {
        sh """
          ${scannerHome}/bin/sonar-scanner \
            -Dsonar.projectKey=customer-support-ticket-system \
            -Dsonar.projectName=Customer-Support-Ticket-System \
            -Dsonar.sources=. \
            -Dsonar.sourceEncoding=UTF-8
        """
      }
    }
  }
}

stage('Build Frontend') {
  steps {
    dir('frontend') {
      sh 'npm run build'
    }
  }
}
```

```
[ec2-user@ip-172-31-14-243 Customer-Support-Ticket-System]$ ls
Jenkinsfile backend frontend k8s
[ec2-user@ip-172-31-14-243 Customer-Support-Ticket-System]$ cat Jenkinsfile
pipeline {
  agent any

  tools {
    nodejs 'NodeJS22'
  }

  environment {
    DOCKERHUB_CREDENTIALS = 'dockerhub-creds'
    DOCKERHUB_USERNAME = 'mudasir18'
    BACKEND_IMAGE = "${DOCKERHUB_USERNAME}/ticket-backend:v1"
    FRONTEND_IMAGE = "${DOCKERHUB_USERNAME}/ticket-frontend:v1"
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Install Backend Dependencies') {
      steps {
        dir('backend') {
          sh 'npm install'
        }
      }
    }

    stage('Install Frontend Dependencies') {
      steps {
        dir('frontend') {
```

```

stage('Build Docker Images') {
    steps {
        sh 'docker build -t $BACKEND_IMAGE backend'
        sh 'docker build -t $FRONTEND_IMAGE frontend'
    }
}

stage('Push Images to Docker Hub') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKER_USER',
                passwordVariable: 'DOCKER_PASS'
            )
        ]) {
            sh '''
            echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin
            docker push $BACKEND_IMAGE
            docker push $FRONTEND_IMAGE
            '''
        }
    }
}

stage('Deploy to Kubernetes') {
    steps {
        sh '''
        kubectl apply -f k8s/
        kubectl rollout restart deployment/ticket-backend
        kubectl rollout restart deployment/ticket-frontend
        '''
    }
}
}

```

```

stage('Deploy to Kubernetes') {
    steps {
        sh '''
        kubectl apply -f k8s/
        kubectl rollout restart deployment/ticket-backend
        kubectl rollout restart deployment/ticket-frontend
        '''
    }
}
}

```

Commit and push the Jenkinsfile:

```

git add Jenkinsfile
git commit -m "Add Jenkins Pipeline"
git push origin main

```

Step 42: Execute the CI/CD Pipeline

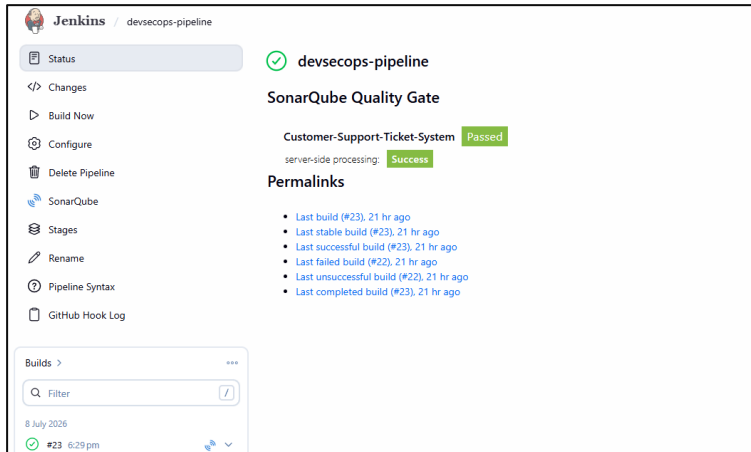
The Jenkins pipeline was executed to automate the complete deployment process.

Pipeline Stages:

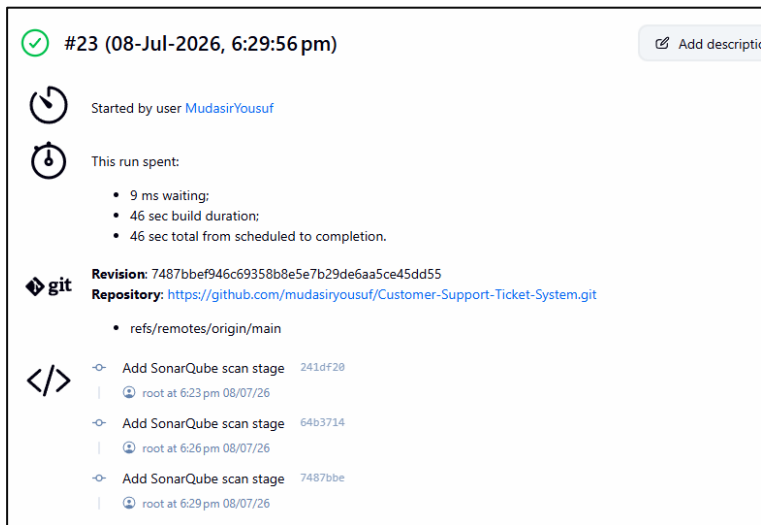
1. Checkout
2. Install Backend Dependencies
3. Install Frontend Dependencies

4. SonarQube Scan
5. Build Frontend
6. Build Docker Images
7. Push Images to Docker Hub
8. Deploy to Kubernetes

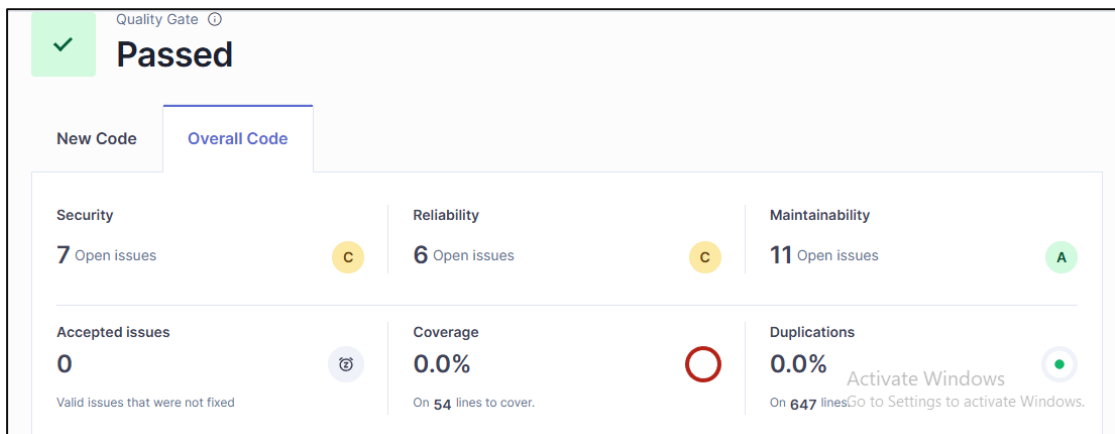
The pipeline completed successfully and deployed the latest application version to the Kubernetes cluster.



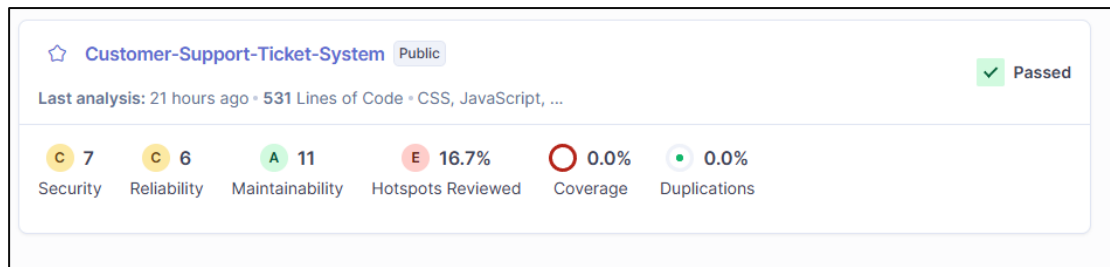
The Jenkins interface shows the 'devsecops-pipeline' with a green checkmark indicating success. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, SonarQube, Stages, Rename, Pipeline Syntax, and GitHub Hook Log. The main panel displays the 'SonarQube Quality Gate' status as 'Passed' for the project 'Customer-Support-Ticket-System'. Below this, a 'Permalinks' section lists various build links. At the bottom, a 'Builds' section shows a list of builds with a filter input and a date '8 July 2026'.



The Jenkins build details page for build #23 (08-Jul-2026, 6:29:56 pm) shows a green checkmark and a button to 'Add description'. The build was started by user 'MudasirYousuf'. The 'This run spent:' section lists: 9 ms waiting, 46 sec build duration, and 46 sec total from scheduled to completion. The 'git' section shows the revision '7487bbef946c69358b8e5e7b29de6aa5ce45dd55' and the repository 'https://github.com/mudasiryousuf/Customer-Support-Ticket-System.git'. The 'Refs' section shows 'refs/remotes/origin/main'. The 'Stages' section shows three stages, each labeled 'Add SonarQube scan stage' with a commit hash and a timestamp.



The SonarQube Quality Gate 'Passed' screen shows a green checkmark and the word 'Passed'. The 'New Code' tab is selected, and the 'Overall Code' tab is also visible. The 'Security' section shows 7 Open issues with a yellow 'C' grade. The 'Reliability' section shows 6 Open issues with a yellow 'C' grade. The 'Maintainability' section shows 11 Open issues with a green 'A' grade. The 'Accepted issues' section shows 0 issues with a green 'A' grade. The 'Coverage' section shows 0.0% coverage with a red 'F' grade and a note 'On 54 lines to cover.' The 'Duplications' section shows 0.0% duplications with a green 'A' grade and a note 'On 647 linesGo to Settings to activate Windows.'



Step 43: Install Helm

Helm was installed as the package manager for Kubernetes.

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
```

Step 44: Add Helm Repository

The Prometheus Community Helm repository was added.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

Step 45: Install Prometheus and Grafana

The **kube-prometheus-stack** Helm chart was used to install Prometheus and Grafana.

```
helm install monitoring prometheus-community/kube-prometheus-stack
```

Verify the installation:

```
kubectl get pods
```

```
kubectl get svc
```

```
kubectl get svc
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0  2/2     Running   4 (45m ago)  20h
monitoring-grafana-5bc65c4644-xkhpk  3/3     Running   6 (44m ago)  20h
monitoring-kube-prometheus-operator-55c9dfccff-wm6tg    1/1     Running   4 (44m ago)  20h
monitoring-kube-state-metrics-6b8f7fb688-864bk         1/1     Running   3 (44m ago)  20h
monitoring-prometheus-node-exporter-582qs              1/1     Running   2 (44m ago)  20h
prometheus-monitoring-kube-prometheus-prometheus-0     2/2     Running   4 (44m ago)  20h
ticket-backend-5ffcdbdb49-6n6cr                       1/1     Running   2 (44m ago)  21h
ticket-frontend-5bf98dffc6-xkmgj                      1/1     Running   2 (44m ago)  21h
```

Step 46: Access Grafana

Grafana was accessed by forwarding the Kubernetes service to port **3000** on the AWS EC2 instance.

Start port forwarding:

```
kubectl port-forward --address 0.0.0.0 svc/monitoring-grafana 3000:80
```

Open Grafana in a web browser:

<http://<EC2-PUBLIC-IP>:3000>

Step 47: Access Prometheus

Prometheus was accessed by forwarding the Kubernetes service to port **9090** on the AWS EC2 instance.

Start port forwarding:

```
kubectrl port-forward --address 0.0.0.0 svc/monitoring-kube-prometheus-prometheus 9090:9090
```

Open Prometheus in a web browser:

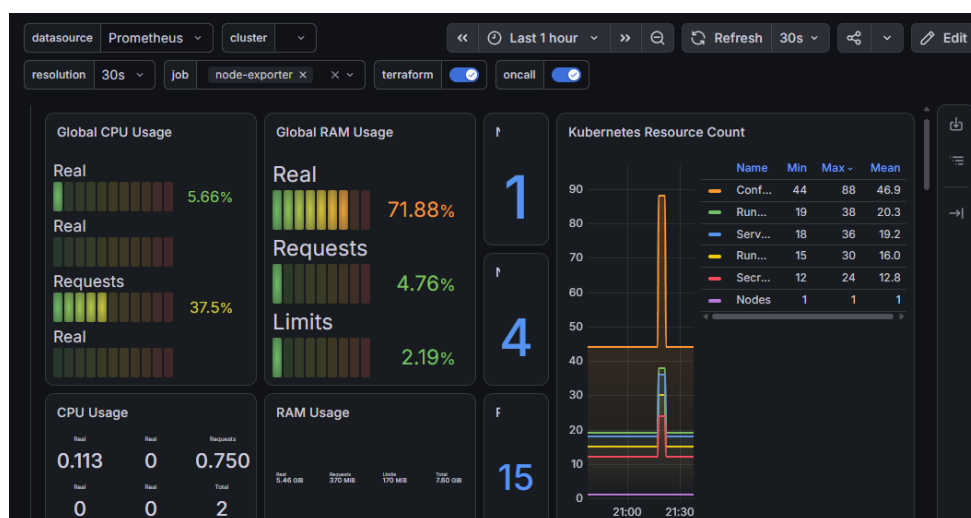
<http://<EC2-PUBLIC-IP>:9090>

Step 48: Monitor the Kubernetes Cluster

Grafana dashboards were used to monitor the Kubernetes cluster and application.

The dashboards provide real-time metrics for:

- CPU Usage
- Memory Usage
- Pod Status
- Deployments
- Node Health
- Network Traffic
- Cluster Performance



Step 49: Verify the Complete DevOps Workflow

The complete pipeline was tested by pushing code changes to the GitHub repository.

Workflow:

1. Push Code to GitHub
2. Jenkins Automatically Starts the Pipeline
3. Source Code is Downloaded
4. SonarQube Performs Code Analysis
5. Docker Images are Built
6. Images are Pushed to Docker Hub
7. Kubernetes Deploys the Latest Images
8. Prometheus Collects Metrics
9. Grafana Displays Real-Time Dashboards

The project was successfully deployed and monitored using a complete end-to-end DevOps CI/CD pipeline.

Troubleshooting

During the implementation of the project, several configuration and deployment issues were identified and resolved to ensure a successful DevOps pipeline.

- Configured Docker permissions for the Jenkins user to execute Docker commands successfully.
- Resolved Jenkins pipeline configuration and plugin compatibility issues.
- Configured SonarQube authentication using a project token for automated code analysis.
- Verified Docker image creation and successful image push to Docker Hub.
- Validated Kubernetes deployments, services, and pod health after deployment.
- Configured Prometheus and Grafana using Helm for Kubernetes monitoring.
- Configured port forwarding to securely access Grafana and Prometheus dashboards from the AWS EC2 instance.
- Verified successful communication between Jenkins, SonarQube, Docker, Kubernetes, Prometheus, and Grafana.
- Performed end-to-end pipeline testing to ensure successful automated build, deployment, and monitoring.

Conclusion

This project successfully demonstrates the implementation of a complete **DevOps CI/CD pipeline** on an **AWS EC2** instance using industry-standard tools and practices. The pipeline automates source code integration, code quality analysis, containerization, image management, Kubernetes deployment, and monitoring.

By integrating **GitHub**, **Jenkins**, **SonarQube**, **Docker**, **Docker Hub**, **Kubernetes (Minikube)**, **Prometheus**, and **Grafana**, the project provides an end-to-end automated workflow that improves deployment efficiency, consistency, and reliability. Real-time monitoring through Prometheus and Grafana enhances system visibility and enables proactive monitoring of the deployed application.

This project demonstrates practical experience with modern DevOps tools and workflows, showcasing the ability to design, implement, deploy, and monitor a complete CI/CD pipeline in a cloud-based environment.